

Contents

Incorporating GitLab into zzcollab: A Technical White Paper	1
Abstract	1
1. Motivation	1
1.1 Why forge neutrality matters for reproducibility	1
1.2 The institutional reality	1
1.3 Scope of the decision	2
2. The problem: implicit GitHub coupling	2
3. Design principles	2
4. Implementation	2
5. Engineering obstacles and their resolution	3
6. Latent defects uncovered	3
7. Verification and limitations	3
8. Future work	4
9. Conclusion	4

Incorporating GitLab into zzcollab: A Technical White Paper

2026-06-23 15:18 PDT

Reference commit: edaf6b0 on branch feat/gitlab-support.

Abstract

This paper documents the design and implementation of first-class GitLab support in zzcollab, a Docker-based framework for reproducible research compendia. Prior to this work the framework was implicitly coupled to GitHub at every layer that touches a git forge: continuous integration, remote-repository creation, container-registry publication, and browser-based cloud launch. We describe why that coupling was a liability for the framework's stated goal of portable reproducibility, present the forge abstraction introduced to remove it, and report the nine-phase implementation, the engineering obstacles encountered, two latent defects uncovered, and the verification performed. We are explicit about what was validated offline and what remains contingent on a live GitLab runner.

1. Motivation

1.1 Why forge neutrality matters for reproducibility

zzcollab organises a research compendium around five pillars: a Dockerfile, an `renv.lock`, an `.Rprofile`, source code, and data. The framework's value proposition is that a compendium remains buildable and verifiable independent of any single vendor or service. A hard dependency on one git forge undercuts that proposition. If the continuous-integration definition, the publication path, and the cloud-launch configuration are all expressed only in GitHub's idioms, then a compendium hosted elsewhere loses automated verification, and an institution that cannot use GitHub cannot adopt the framework without bespoke rework.

1.2 The institutional reality

GitLab is widely deployed in exactly the settings zzcollab targets. Many universities and research organisations operate self-hosted GitLab instances for data-governance and sovereignty reasons: source code, continuous integration, and the container registry remain inside the institution's own infrastructure. For such users a GitHub-only framework is not a preference mismatch but an adoption blocker. GitLab also offers a tightly integrated single-platform story (repository, CI, container registry, and Workspaces in one product), and its CI provides registry authentication automatically through built-in variables, which removes a class of credential-handling that the GitHub plus Docker Hub path requires.

1.3 Scope of the decision

Supporting GitLab was therefore framed not as a cosmetic addition but as the removal of an architectural assumption. The work was scoped through a prior planning document (`docs/gitlab-support-plan.md`) and four explicit design decisions taken with the maintainer: a project selects a single forge; remote creation uses the `glab` CLI with self-hosted support; GitLab Workspaces is supported for cloud launch; and the GitLab Container Registry is included, generalising the registry layer rather than adding a third special case.

2. The problem: implicit GitHub coupling

An exhaustive inventory located the coupling across eight categories:

1. Two GitHub Actions workflow templates (`r-package.yml`, `render-report.yml`).
2. Three artifact-detection sites that inferred ‘CI is on’ from the presence of `.github/workflows/r-package.yml` (status, the toggle wizard, and the doctor version check).
3. A `cmd_github` command built entirely on the `gh` CLI and hardcoded `https://github.com/...` remote URLs.
4. Nine `gh` invocations spread across remote creation, account validation, and username detection.
5. Configuration keys (`github.account`, `visibility`, `branch`) with no forge selector.
6. A cloud-launch path that installed only a Codespaces devcontainer.
7. Documentation that presented GitHub as the only option.
8. Help text and command dispatch that registered only GitHub verbs.

Crucially, there was no concept of a ‘forge’ anywhere. GitHub was not a configured choice; it was a buried assumption.

3. Design principles

The implementation was governed by four principles.

- **Single explicit forge.** A new `forge` key takes `github`, `gitlab`, or `none`, defaulting to `github` for backward compatibility. Consumers read it as `${CONFIG_FORGE:-github}`, so existing projects behave exactly as before until they opt in.
- **Graceful degradation.** Every external-CLI dependency (`glab`, and the existing `gh`) is guarded so its absence yields a clear message rather than a failure. CI, registry, and Workspaces configuration are generated with no CLI present; only live repository creation needs `glab`.
- **Centralisation over duplication.** Forge-specific logic was collected into small shared helpers (`zxc_ci_forge`, `forge_user`, `forge_account_exists`) so detection and account handling have one implementation each, consumed identically by status, the wizard, doctor, and the remote commands.
- **Reuse of existing machinery.** The GitLab templates pass through the same `envsubst`-based renderer, version-stamp scheme, and `zxc update` regeneration registry as the GitHub templates, so they are maintained by the same mechanisms rather than a parallel path.

4. Implementation

The work proceeded in nine phases, each verified before the next.

- **P1, configuration.** Added `forge`, `gitlab.account`, `gitlab.host`, and the GitLab visibility and branch keys, with a validator that rejects any forge outside the permitted set. The friendly CLI keys derive automatically from the YAML paths, so no aliases were required.
- **P2, continuous integration.** Authored a single, self-adapting `.gitlab-ci.yml` that replaces the two GitHub workflows. Backend selection (Nix flake, Docker, `renv`, or `DESCRIPTION`) is expressed through rule `exists:` clauses rather than the GitHub detect-job indirection, and a path-filtered render stage mirrors the original behaviour.
- **P3, install, detect, and remove.** Introduced `zxc_ci_forge` and made installation, the three detection sites, removal, and the `zxc update` managed-file registry forge-aware, so a project regenerates only the CI it actually carries.

- **P4, remote creation.** Added `cmd_gitlab` and `cmd_rm_gitlab` over the `glab` CLI, honouring a self-hosted host and public, private, or internal visibility, and extracted the shared initial-commit logic used by both forges.
- **P5, convenience detection.** Added forge-aware `forge_user` and `forge_account_exists` helpers, adopted them where `gh` was previously hardcoded, and removed the duplication introduced in P4.
- **P6, container registry.** Generalised the publish command to honour the configured registry for Docker Hub, the GitHub Container Registry, the GitLab Container Registry, and self-hosted GitLab registries, and defaulted to the GitLab registry when the forge is GitLab.
- **P7, cloud launch.** Added a GitLab Workspaces devfile as the forge-default cloud-launch platform, with detection and removal extended accordingly.
- **P8, interactive surface.** Added a forge picker to the toggle wizard, rendered through the cursor-driven information-box chooser developed earlier in this effort, made the interactive menu forge-aware, and renamed the publish verb to the forge-neutral `ztc push` while retaining `dockerhub` as an alias.
- **P9, documentation.** Added a consolidated GitLab section to the user guide and updated the README command surface and configuration keys.

5. Engineering obstacles and their resolution

Several problems were specific to translating GitHub idioms into GitLab’s execution model.

- **Template rendering versus CI variables.** `ztcollab` renders templates through `envsubst` with an allowlist. GitLab pipelines are dense with `$CI_*` variables that must survive rendering untouched. We confirmed that an allowlisted `envsubst` substitutes only named variables and leaves all others literal, and observed the discipline of avoiding shell-variable names that collide with the allowlist inside script blocks.
- **Backend selection without negative conditions.** GitLab’s `rules: exists:` has no native negation. The `nix`, `Docker`, and `host` paths were expressed with ordered `when:` `never` guards followed by a positive fallback, reproducing the GitHub matrix without a detect job.
- **Docker-in-Docker isolation.** The GitHub render mounts the working tree into the project image. GitLab’s `dind` daemon cannot see the runner’s checkout, so a volume mount would be empty. The render job instead layers the source onto the environment image, runs inside it, and copies the rendered output back for artifact collection.
- **Cache locality.** GitLab caches only paths inside the project directory, whereas the GitHub workflow caches a home-directory `renv` cache. The GitLab job redirects the `renv` cache into the project tree so it is cacheable.

6. Latent defects uncovered

The registry generalisation surfaced two pre-existing defects unrelated to GitLab. First, the publish command ignored the `docker.registry` configuration entirely and always pushed to Docker Hub. Second, the status display printed a hardcoded registry string rather than reading the configuration. Both were corrected as part of the registry work. In addition, a regression introduced during the convenience-detection phase, where a not-found account check would have aborted the configuration wizard under `set -e`, was caught during verification and fixed before integration.

7. Verification and limitations

The work was verified at every phase by direct execution of the deterministic components: configuration round-trips, forge detection, template installation and version-stamp extraction, registry-reference construction across all four registries, the CLI-absent guards, and the help and dispatch surface. All shell sources pass `shellcheck` at warning severity.

The runtime surfaces that depend on external services were not exercised, because no `glab` binary, GitLab instance, or registry credentials were available in the development environment. Specifically, an end-to-end CI pipeline run (notably the Docker-in-Docker render job), live repository creation and push through `glab`, a

GitLab Workspaces launch, and registry pushes are written to documented interfaces but remain unvalidated. These should be confirmed against a real GitLab project before the feature is considered production-ready. This limitation is stated plainly rather than implied: offline structural validation is not a substitute for a live run.

8. Future work

Two items were deliberately deferred. The interactive configuration setup does not yet present a dedicated forge or GitLab-account prompt; the toggle wizard is currently the forge entry point, and the existing account-check helper is ready to support such a prompt. The publish-verb rename retained the `dockerhub` alias; a future release may deprecate it once the forge-neutral name is established.

9. Conclusion

Adding GitLab to zzcollab was, in substance, the replacement of an unstated assumption with an explicit, validated choice. By introducing a single forge abstraction and routing continuous integration, remote creation, registry publication, and cloud launch through it, the framework now serves the self-hosted and GitLab-based research settings it previously excluded, without disturbing existing GitHub users. The work also repaid technical debt in the registry layer. What remains is the honest and necessary step of validating the GitLab runtime paths on live infrastructure.

Rendered on 2026-06-23 at 15:18 PDT. Source: ~/prj/sfw/07-zzcollab/zzcollab/docs/gitlab-integration-whitepaper.md